HDOC DAY-72

Develop a C program to manage student attendance using a dynamically resizable matrix. The program must use pointers for matrix handling and must not use a fixed-size 2D array. Each row of the matrix represents one student, and each column represents one working day. Use 1 to indicate "Present" and 0 to indicate "Absent". The program must:

- Accept the initial number of students and working days.
- Use malloc() to dynamically allocate memory for the initial attendance matrix.
- Accept the attendance details of the initial students.
- Allow additional students to be added dynamically.
- Use realloc() to increase the matrix size when new students are added.
- Accept only 0 or 1 as attendance input.
- Calculate and display the attendance percentage of each student.
- Use separate functions for:
    - Creating the initial matrix
    - Recording attendance
    - Adding students
    - Displaying attendance records
    - Calculating attendance percentages
    - Releasing allocated memory
- Handle invalid input and memory-allocation failure appropriately.
- Release all dynamically allocated memory using free() before program termination.

Algorithm:

1. Start.
2. Input the initial number of students.
3. Validate that the number of students is greater than 0.
4. Input the number of working days.
5. Validate that the number of working days is greater than 0.

6. Call createMatrix() to:
    - Dynamically allocate the array of row pointers using malloc().
    - Dynamically allocate each student's attendance row using malloc().
7. Check whether memory allocation was successful.
8. Call recordAttendance() to enter attendance for each student.
9. For every attendance value:
    - Accept 0 for absent.
    - Accept 1 for present.
    - Reject any other value and ask again.
10. Call displayAttendance() to display the initial attendance matrix.
11. Input the number of new students to add.
12. If new students are added:
    - Store the old number of students.
    - Increase the total number of students.
    - Call addStudents().
    - Use realloc() to increase the array of row pointers.
    - Use malloc() to create rows for the new students.
13. Call recordAttendance() to enter attendance for the newly added students.
14. Call displayAttendance() to display the updated attendance matrix.
15. Call calculatePercentage().
16. For each student:
    - Count the number of days present.
    - Calculate:
17. Display each student's attendance percentage.
18. Call freeMatrix() to release every dynamically allocated row.
19. Free the array of row pointers.
20. Stop.

- **C Program:**

```
#include <stdio.h>
#include <stdlib.h>

/* Function declarations */
int **createMatrix(int students, int days);
void recordAttendance(int **attendance, int start, int end, int days);
int **addStudents(int **attendance, int oldStudents, int newStudents, int days);
void displayAttendance(int **attendance, int students, int days);
void calculatePercentage(int **attendance, int students, int days);
void freeMatrix(int **attendance, int students);

int main()
{
    int **attendance;
    int students, days;
    int add, oldStudents;

    /* Input number of students */
    do
    {
        printf("Enter initial number of students: ");
        scanf("%d", &students);

        if (students <= 0)
            printf("Invalid input! Number of students must be positive.\n");

    } while (students <= 0);

    /* Input number of working days */
    do
    {
        printf("Enter number of working days: ");
        scanf("%d", &days);

        if (days <= 0)
            printf("Invalid input! Number of days must be positive.\n");

    } while (days <= 0);

```

```
    /* Create initial matrix */
    attendance = createMatrix(students, days);

    if (attendance == NULL)
    {
       printf("Memory allocation failed.\n");
       return 1;
    }

    /* Record attendance of initial students */
    printf("\nEnter attendance for initial students:\n");
    recordAttendance(attendance, 0, students, days);

    printf("\nInitial Attendance Record:\n");
    displayAttendance(attendance, students, days);

    /* Add students */
    do
    {
       printf("\nEnter number of new students to add: ");
       scanf("%d", &add);

       if (add < 0)
          printf("Invalid input! Enter 0 or a positive number.\n");

    } while (add < 0);

    if (add > 0)
    {
       oldStudents = students;
       students = students + add;

       attendance = addStudents(attendance, oldStudents,
                          students, days);

       if (attendance == NULL)
       {
          printf("Memory reallocation failed.\n");
          return 1;
```

```
        }

        printf("\nEnter attendance for new students:\n");
        recordAttendance(attendance, oldStudents,
                    students, days);
    }

    /* Display final attendance */
    printf("\nFinal Attendance Record:\n");
    displayAttendance(attendance, students, days);

    /* Calculate percentages */
    printf("\nAttendance Percentage:\n");
    calculatePercentage(attendance, students, days);

    /* Release memory */
    freeMatrix(attendance, students);

    return 0;
}


/* Create attendance matrix using malloc() */
int **createMatrix(int students, int days)
{
    int **matrix;
    int i;

    matrix = (int **)malloc(students * sizeof(int *));

    if (matrix == NULL)
        return NULL;

    for (i = 0; i < students; i++)
    {
        *(matrix + i) = (int *)malloc(days * sizeof(int));

        if (*(matrix + i) == NULL)
        {
```

```
            int j;

            for (j = 0; j < i; j++)
               free(*(matrix + j));

            free(matrix);
            return NULL;
         }
      }

      return matrix;
   }


   /* Record attendance */
   void recordAttendance(int **attendance, int start,
                  int end, int days)
   {
      int i, j;
      int value;

      for (i = start; i < end; i++)
      {
         printf("\nStudent %d:\n", i + 1);

         for (j = 0; j < days; j++)
         {
            do
            {
               printf("Day %d (1 = Present, 0 = Absent): ",
                     j + 1);

               scanf("%d", &value);

               if (value != 0 && value != 1)
               {
                  printf("Invalid input! Enter only 0 or 1.\n");
               }
```

```
            } while (value != 0 && value != 1);

            *(*(attendance + i) + j) = value;
        }
    }
}


/* Add new students using realloc() */
int **addStudents(int **attendance, int oldStudents,
                  int newStudents, int days)
{
    int **temp;
    int i;

    temp = (int **)realloc(attendance,
                           newStudents * sizeof(int *));

    if (temp == NULL)
    {
        printf("Memory reallocation failed.\n");
        return NULL;
    }

    attendance = temp;

    for (i = oldStudents; i < newStudents; i++)
    {
        *(attendance + i) =
            (int *)malloc(days * sizeof(int));

        if (*(attendance + i) == NULL)
        {
            printf("Memory allocation failed for Student %d.\n",
                   i + 1);

            /*
             * Free rows that were successfully allocated
             * for new students.
```

```
            */
            while (--i >= oldStudents)
               free(*(attendance + i));

            return NULL;
         }
      }

      return attendance;
   }


   /* Display attendance records */
   void displayAttendance(int **attendance,
                          int students, int days)
   {
      int i, j;

      printf("\nStudent\t");

      for (j = 0; j < days; j++)
         printf("Day%d\t", j + 1);

      printf("\n");

      for (i = 0; i < students; i++)
      {
         printf("%d\t", i + 1);

         for (j = 0; j < days; j++)
         {
            printf("%d\t",
                 *(*(attendance + i) + j));
         }

         printf("\n");
      }
   }
```

```
/* Calculate attendance percentage */
void calculatePercentage(int **attendance,
                         int students, int days)
{
   int i, j;
   int present;
   float percentage;

   for (i = 0; i < students; i++)
   {
      present = 0;

      for (j = 0; j < days; j++)
      {
         present += *(*(attendance + i) + j);
      }

      percentage = (present * 100.0f) / days;

      printf("Student %d: %.1f%%\n",
           i + 1, percentage);
   }
}


/* Release dynamically allocated memory */
void freeMatrix(int **attendance, int students)
{
   int i;

   for (i = 0; i < students; i++)
   {
      free(*(attendance + i));
   }

   free(attendance);
}
```

Test Case Table:

Test Case	Initial Students	Working Days	Initial Attendance	Students Added	New Attendance	Expected Result
1	3	3	1 0 0 , 1 0 1, 1 1 1	3	1 0 0 , 1 1 1 , 1 0 1	Percentage: 33.3%, 66.7%, 33.3%, 100%, 66.7%
2	2	3	1 1 1, 0 0 0	0	-	Percentage: 100% , 0%
3	1	4	1 1 0 0	0	-	Percentage: 50%
4	0	3	-	-	-	Invalid number of students; ask again
5	3	0	-	-	-	Invalid number of working days; ask again
6	1	3	User enters 1 2 0	-	-	2 is rejected; program asks for attendance again
7	2	2	1 0 , 0 1	2	1 1 , 0 0	50% , 50% , 100% , 0%

Compilation and Execution:

The following program has been compiled and executed successfully.

Test Case 1:

```
└─$ ./a.out
Enter initial number of students: 3
Enter number of working days: 3

Enter attendance for initial students:

Student 1:
Day 1 (1 = Present, 0 = Absent): 1 0 0
Day 2 (1 = Present, 0 = Absent): Day 3 (1 = Present, 0 = Absent):
Student 2:
Day 1 (1 = Present, 0 = Absent): 1 0 1
Day 2 (1 = Present, 0 = Absent): Day 3 (1 = Present, 0 = Absent):
Student 3:
Day 1 (1 = Present, 0 = Absent): 1 1 1
Day 2 (1 = Present, 0 = Absent): Day 3 (1 = Present, 0 = Absent):
Initial Attendance Record:

Student Day1    Day2    Day3
1       1       0       0
2       1       0       1
3       1       1       1

Enter number of new students to add: 3

Enter attendance for new students:

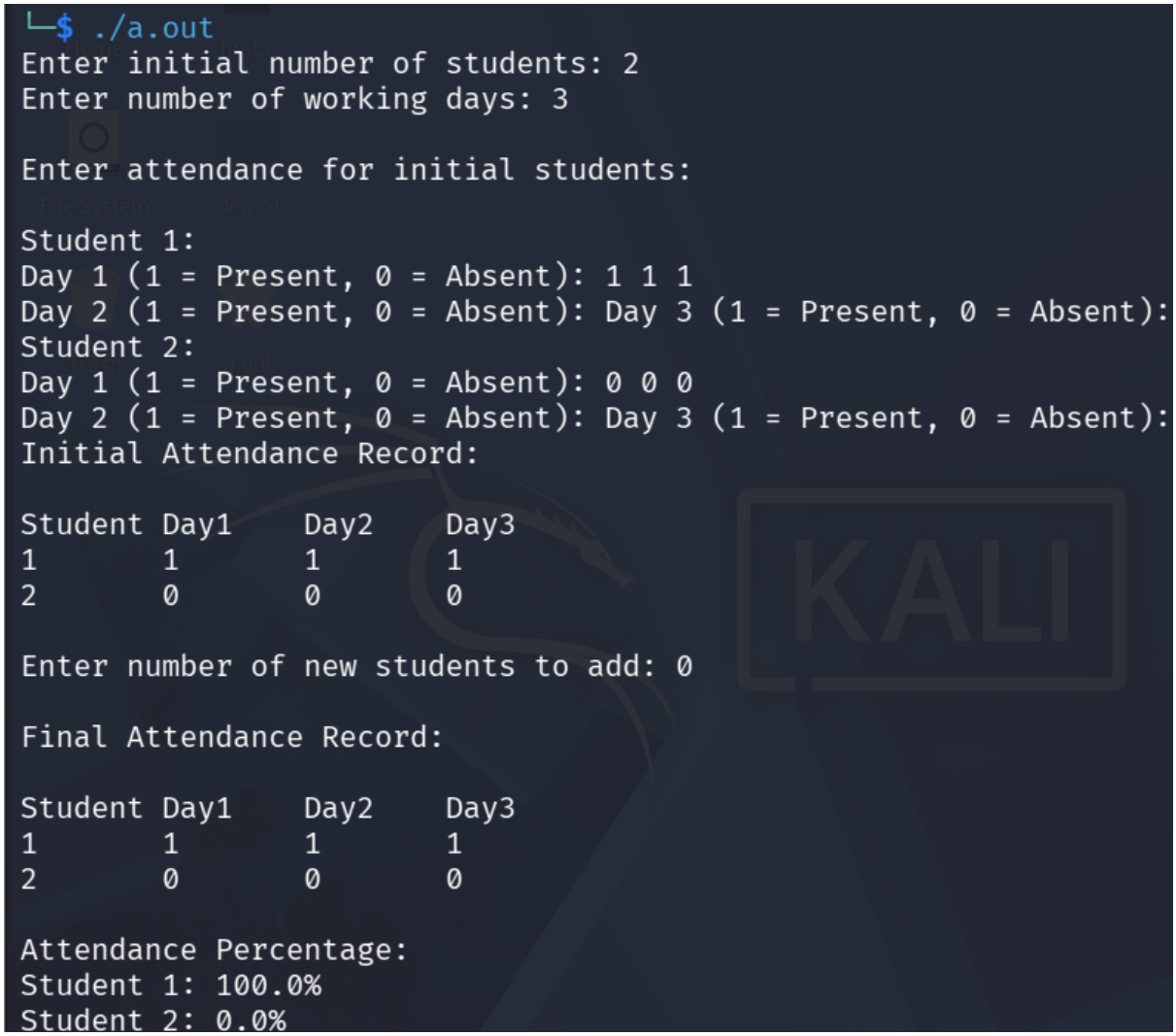
Student 4:
Day 1 (1 = Present, 0 = Absent): 1 0 0
Day 2 (1 = Present, 0 = Absent): Day 3 (1 = Present, 0 = Absent):
Student 5:
Day 1 (1 = Present, 0 = Absent): 1 1 1
Day 2 (1 = Present, 0 = Absent): Day 3 (1 = Present, 0 = Absent):
```

```
Student 6:
Day 1 (1 = Present, 0 = Absent): 1 0 1
Day 2 (1 = Present, 0 = Absent): Day 3 (1 = Present, 0 = Absent):
Final Attendance Record:

Student Day1    Day2    Day3
1       1       0       0
2       1       0       1
3       1       1       1
4       1       0       0
5       1       1       1
6       1       0       1

Attendance Percentage:
Student 1: 33.3%
Student 2: 66.7%
Student 3: 100.0%
Student 4: 33.3%
Student 5: 100.0%
Student 6: 66.7%
```

Test Case 2:



Test Case 3:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter initial number of students: 1
Enter number of working days: 4

Enter attendance for initial students:

Student 1:
Day 1 (1 = Present, 0 = Absent): 1 1 0 0
Day 2 (1 = Present, 0 = Absent): Day 3 (1 = Present,
Present, 0 = Absent):
Initial Attendance Record:

Student Day1    Day2    Day3    Day4
1       1       1       0       0

Enter number of new students to add: 0

Final Attendance Record:

Student Day1    Day2    Day3    Day4
1       1       1       0       0

Attendance Percentage:
Student 1: 50.0%
```

Test Case 4:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter initial number of students: 0
Invalid input! Number of students must be positive
```

Test Case 5:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter initial number of students: 3
Enter number of working days: 0
Invalid input! Number of days must be positive.
```

Test Case 6:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter initial number of students: 2
Enter number of working days: 2

Enter attendance for initial students:

Student 1:
Day 1 (1 = Present, 0 = Absent): 1 0
Day 2 (1 = Present, 0 = Absent):
Student 2:
Day 1 (1 = Present, 0 = Absent): 0 1
Day 2 (1 = Present, 0 = Absent):
Initial Attendance Record:

Student Day1    Day2
1       1       0
2       0       1

Enter number of new students to add: 2

Enter attendance for new students:

Student 3:
Day 1 (1 = Present, 0 = Absent): 1 1
Day 2 (1 = Present, 0 = Absent):
Student 4:
Day 1 (1 = Present, 0 = Absent): 0 0
Day 2 (1 = Present, 0 = Absent):
Final Attendance Record:
```

```
Student Day1    Day2
1       1       0
2       0       1
3       1       1
4       0       0

Attendance Percentage:
Student 1: 50.0%
Student 2: 50.0%
Student 3: 100.0%
Student 4: 0.0%
```